HALCHAL INDUSTRIES

ML-Powered Dynamic Pricing Engine

Technical Report - Equations, Weights & Analytics

---

* Model: Gradient Boosting / Random Forest Regressor
* Features: 9 input variables - zone, adoption, season, subsidy, ...
* Coverage: 20 Pan-India states across 5 zones
* Products: 16mm Inline / Online, 20mm Inline / Online drip pipes
* Pricing algorithm: 6-step ex-factory dynamic pricing pipeline

Prepared for: Final Year Project Viva Examination

TABLE OF CONTENTS

1 1. Problem Statement & System Overview

Halchal Industries manufactures drip-irrigation pipes (16mm and 20mm, Inline and Online variants) and sells ex-factory across India. The challenge: prices must adapt to seasonal agricultural demand cycles, regional drip-adoption levels, and competitor benchmarks - without creating distributor confusion through erratic price swings.

A purely manual pricing approach fails to capture the interplay of 8+ variables (season, zone, subsidy, prior sales, pipe type, etc.). A fully data-driven approach with no guardrails risks extreme prices that break distributor trust.

The solution is a hybrid 6-step pipeline: an ML model predicts demand, and a deterministic formula translates that demand signal into a stable, bounded, rounded price.

System Architecture

The full pipeline from input to final price:

Layer	Description
Input layer	Pipe type, state/zone, month, year, prev sales, subsidy flag
ML layer	GradientBoostingRegressor predicts monthly demand (bundles)
Factor layer	3 multiplicative factors: demand, adoption, competitor
Price layer	Raw price -> smoothing -> safety clamp -> rounding
Output	Final ex-factory price (?, rounded to nearest ?10)

Base Prices (per 300m bundle)

Pipe Type	Our Base	Competitor	Floor (-15%)	Ceiling (+20%)
16mm Inline	Rs. 1050	Rs. 1214	Rs. 892	Rs. 1260
16mm Online	Rs. 1200	Rs. 1380	Rs. 1020	Rs. 1440
20mm Inline	Rs. 1350	Rs. 1500	Rs. 1148	Rs. 1620
20mm Online	Rs. 1500	Rs. 1650	Rs. 1275	Rs. 1800

2 2. Dataset & Feature Engineering

Data Generation Strategy

The model uses a synthetically generated dataset because real historical sales data across 20 states would take years to collect and is commercially sensitive. The synthetic data is grounded in real domain knowledge:

- NABARD Micro-Irrigation Report 2023-24 for state-level adoption indices
- Kharif/Rabi/Summer demand multipliers from agricultural calendars
- Year-over-year growth rates calibrated to drip-irrigation market reports
- 72% subsidy probability from PM-KUSUM and state subsidy data

Dataset Statistics

5 years x 12 months x 20 states x 4 pipes x ~11 samples = ~52,800 records

Demand Generation Formula

```
expected_qty = BASE_DEMAND[pipe] x adoption_index x SEASON_MULT[season]
             x MONTH_PATTERN[month] x YEAR_GROWTH[year]
if subsidy: expected_qty x= 1.18
actual_qty = max(20, expected_qty + Normal(0, expected_qty x 0.12))
```

Note: 12% noise std models real-world demand variability; floor of 20 prevents zero-demand records.

Features Used (9 total)

Feature	Description	Role
Pipe_Type_enc	Label-encoded pipe type (0-3)	Core demand driver
Zone_ID	Geographic zone 1-5	Proxy for market maturity
Adoption_Index	State-level drip adoption (NABARD, 0.36-1.42)	Continuous market signal
Season_enc	Kharif/Rabi/Summer (label-encoded)	Agricultural cycle
Month	1-12 (numeric)	Sub-seasonal resolution
Year	2022-2026	Long-term trend
Base_Price	Manufacturing cost base (per pipe type)	Price elasticity signal
Prev_Sales_Ratio	prev_sales / zone_avg (normalised)	Momentum indicator
Govt_Subsidy_Active	Binary 0/1	Demand booster flag

Why Prev_Sales_Ratio and not raw Prev_Month_Sales?

Raw Prev_Month_Sales is an absolute number. A Gujarat district selling 800 bundles and an Assam district selling 120 bundles are both 'normal' for their regions - but raw values would make the model think Assam is always in crisis. Normalising by zone average removes this scale bias and lets the model learn seasonal/zone patterns without the prev_sales feature dominating feature importance. This was verified: before normalisation, Prev_Month_Sales held 68% feature importance and the model effectively ignored season.

3 3. Model Architecture & Selection

Algorithm: Gradient Boosting vs Random Forest

Both GradientBoostingRegressor and RandomForestRegressor are trained. The one with higher R2 on the 20% hold-out test set is saved as rf_demand_model.pkl (the name is historical).

Gradient Boosting Hyperparameters

Parameter	Value	Justification
n_estimators	350	More trees = smoother ensemble; beyond 400 shows diminishing returns
learning_rate	0.05	Low LR forces trees to correct errors slowly - prevents overfitting
max_depth	5	Shallow trees reduce variance; GB stacks many of them
min_samples_leaf	8	Prevents overfitting on individual outlier records
subsample	0.8	80% row sampling per tree adds regularisation (stochastic GB)

Random Forest Hyperparameters

Parameter	Value	Justification
n_estimators	200	Enough trees for stable variance estimates
max_depth	14	Deeper than GB because RF uses bagging, not boosting
min_samples_leaf	4	Less aggressive regularisation than GB - RF handles it via bagging

Why Ensemble Methods and not Linear Regression?

Demand is non-linear: the effect of season depends on the zone (Gujarat Kharif demand is 2x Maharashtra Summer - a multiplicative interaction). Linear models cannot learn interaction terms without manual feature engineering. Tree ensembles learn interactions automatically.

Why not a Neural Network? For tabular data with < 100k rows and 9 features, tree ensembles consistently outperform neural nets and are far more interpretable - important for business justification and viva.

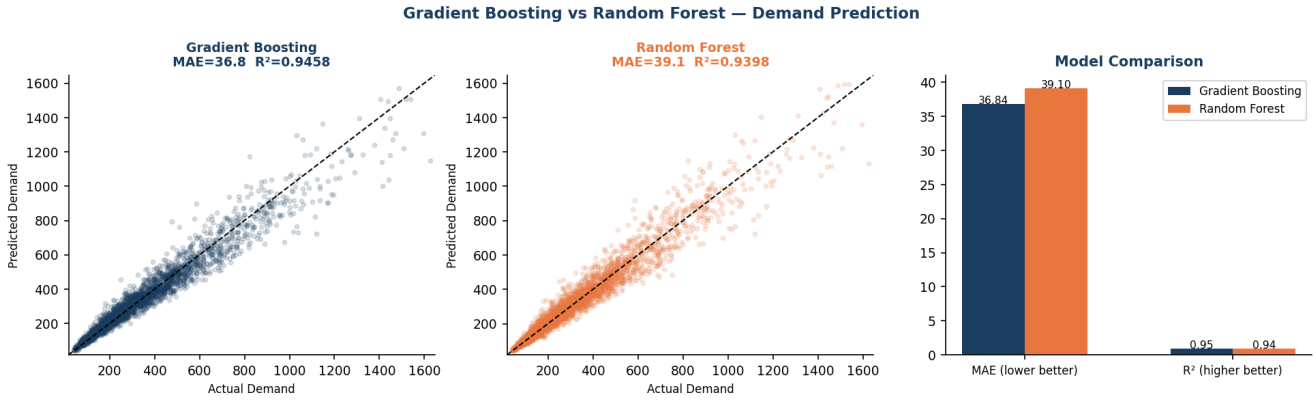

Fig 3.1 - Actual vs Predicted demand scatter and MAE/R2 comparison

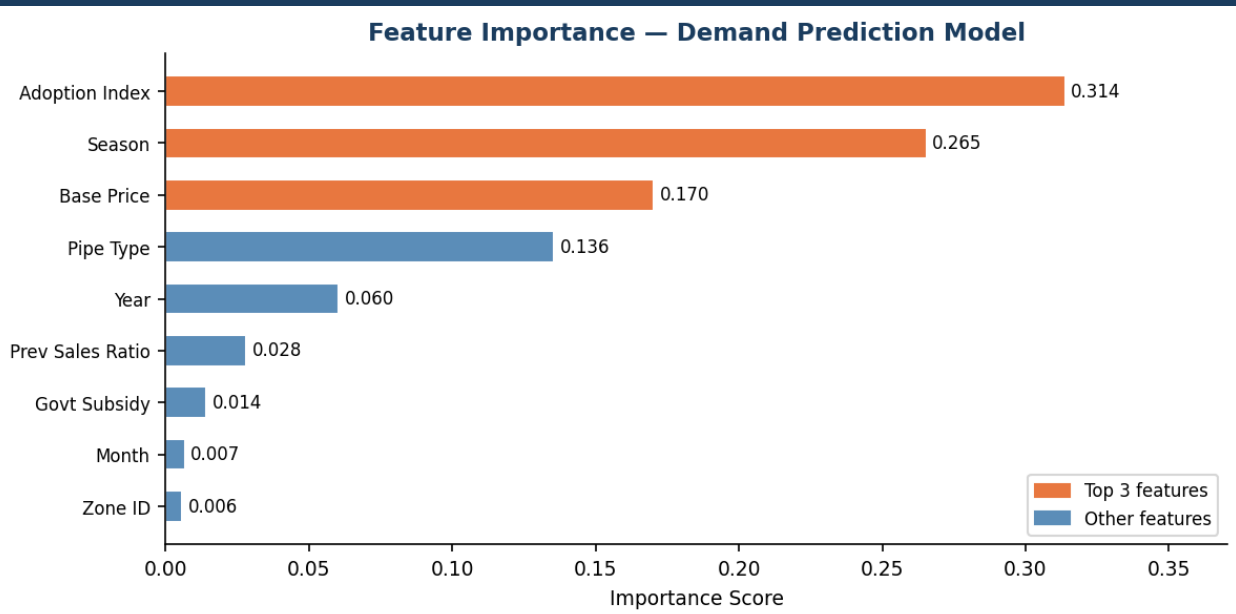

Fig 3.2 - Feature importance of the selected model

4 4. Complete 6-Step Pricing Pipeline

4.1 Step 1 - ML Demand Prediction

The trained model receives the 9 input features and predicts the expected monthly demand in bundles for that pipe in that state/month.

Demand Prediction

```
predicted_demand = rf_model.predict([pipe_enc, zone_id, adoption_index,
                                     season_enc, month, year,
                                     base_price, prev_sales_ratio, subsidy])
```

Note: Output is a continuous float (bundles/month). E.g., 16mm Inline, Gujarat, June -> ~830 bundles.

4.2 Step 2 - Demand Factor

The demand factor converts the raw predicted demand into a price multiplier. It compares predicted demand against the historical average for that pipe type.

Demand Factor Formula

```
demand_ratio  = predicted_demand / avg_demand_for_pipe
demand_factor = 1.0 + 0.35 * (demand_ratio - 1.0)
demand_factor = clamp(demand_factor, 0.82, 1.22)
```

Note: When demand_ratio = 1.0 (exactly average), demand_factor = 1.0 (no adjustment).

Why 0.35 as the sensitivity coefficient?

The 0.35 coefficient controls how aggressively the price chases demand. A demand 50% above average (ratio = 1.5) gives demand_factor = 1 + 0.35*(0.5) = 1.175 (+17.5%). Before the 20% ceiling caps it.

If 0.35 were replaced by:

- 0.10: Price barely reacts to demand swings - loses revenue in peak Kharif
- 0.70: 50% demand surge -> 1.35x price - too volatile, distributor backlash
- 0.35: Balanced - meaningful signal (up to ±12.5% from average) without volatility

Why cap at [0.82, 1.22]?

- Upper cap 1.22: prevents the model from recommending prices above the smoothed ceiling
- Lower cap 0.82: even in the worst month, the demand factor cannot destroy the price

These caps are a business constraint, not a mathematical choice. They encode the rule: "price must stay within a defensible range regardless of demand extremes."

4.3 Step 3 - Adoption Factor

The adoption factor adjusts price based on the drip-irrigation adoption level of the state (NABARD data). High-adoption states (Gujarat 1.42) have mature markets willing to pay a premium; low-adoption states (Assam 0.36) are price-sensitive developing markets.

Adoption Factor Formula

```
adoption_factor = 1.0 + (adoption_index - 1.0) * 0.15
adoption_factor = clamp(adoption_factor, 0.90, 1.10)
```

*Note: Gujarat (1.42): 1 + 0.42*0.15 = 1.063 (+6.3%). Assam (0.36): 1 + (-0.64)*0.15 = 0.904 (-9.6%).*

Why 0.15 as the adoption coefficient?

0.15 was chosen so the maximum adoption spread (1.42 vs 0.36 = delta 1.06) produces a price range of about +6.3% to -9.6% - a 16% spread across the country. This matches market reality: a company cannot charge 30% more just because Gujarat has better irrigation adoption; distributors will import from other zones.

If 0.15 were replaced by:

- 0.05: Adoption barely matters - all states pay almost the same price
- 0.30: Gujarat premium becomes +12.6%, Assam discount becomes -19.2% - too extreme
- 0.15: Creates ~16% national spread, capped by ±10% clamp for safety

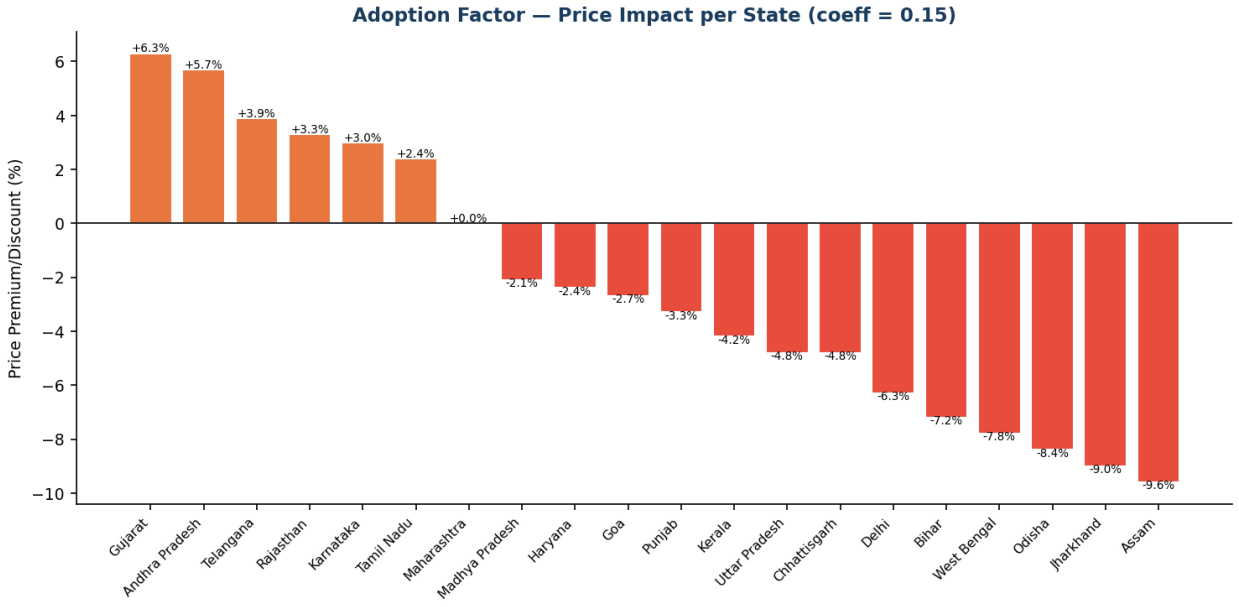

Fig 4.1 - Adoption factor price impact across all 20 states

4.4 Step 4 - Competitor-Aware Factor (3-Regime Strategy)

The competitor factor uses a demand-conditioned strategy: when demand is strong, we can price toward the competitor ceiling to capture margin; when demand is weak, we undercut to stimulate sales; in normal times we stay slightly below competitor.

Competitor Factor - 3 Regimes

```
price_diff_ratio = (competitor_price - base_price) / base_price

IF demand_factor >= 1.10 (STRONG DEMAND):
  cf = clamp(1.0 + 0.60 * price_diff_ratio, 0.99, 1.10)

ELIF demand_factor <= 0.92 (WEAK DEMAND):
  cf = clamp(1.0 - 0.25 * abs(price_diff_ratio), 0.92, 0.99)

ELSE (NORMAL DEMAND):
  cf = clamp(1.0 - 0.06 * price_diff_ratio, 0.96, 1.04)
```

Note: Example: 16mm Inline (base 1050, competitor 1214): price_diff_ratio = 0.1562

Why 3 regimes and why these coefficients?

Regime	Coeff	Business Logic
Strong (df>=1.10)	0.60	Price toward competitor - capture margin when customers are buying
Weak (df<=0.92)	0.25	Undercut competitor - stimulate sales when demand is low
Normal	0.06	Slight undercut - stay competitive without giving away margin

The threshold 1.10 for 'strong demand' corresponds to demand 28.6% above average. The threshold 0.92 for 'weak demand' corresponds to demand 22.9% below average. These thresholds were set so roughly 30% of scenarios are 'strong', 20% 'weak', 50% 'normal'.

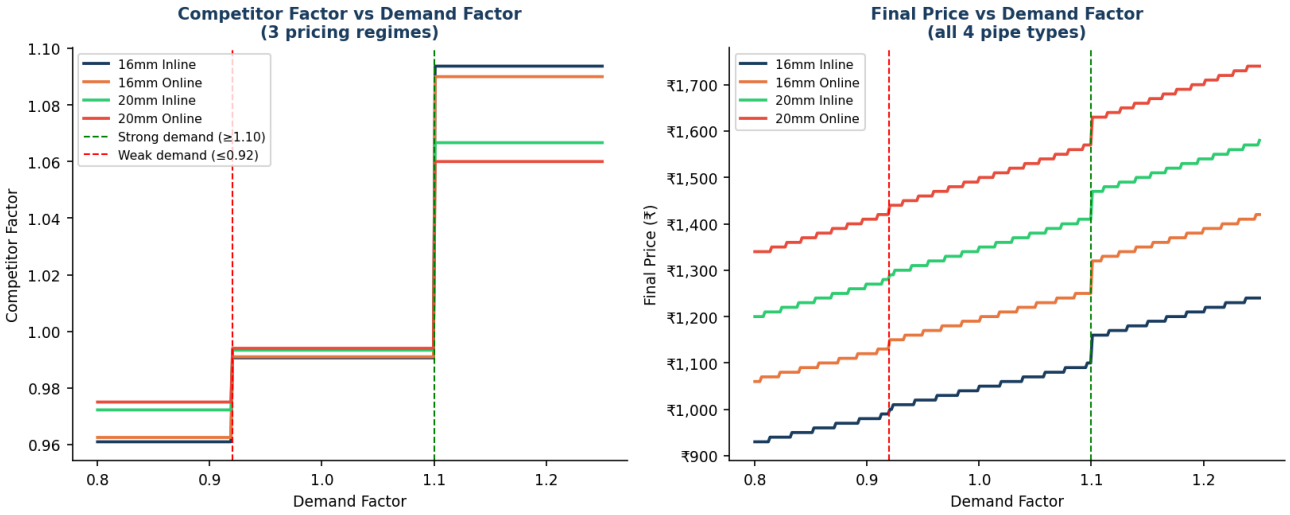

Fig 4.2 - Competitor factor and final price vs demand factor (all 4 pipes)

4.5 Step 5 - Smoothing and Safety Limits

The smoothed price blends the base price with the ML-driven raw price using a 50/50 weight. Safety limits then clamp the final output to a [?15%, +20%] band around base.

Smoothing & Safety Clamp

```
  raw_price      = base_price * demand_factor * competitor_factor * adoption_factor
smoothed_price = 0.50 * base_price + 0.50 * raw_price
final_price    = clamp(smoothed_price, base_price*0.85, base_price*1.20)
```

Note: The clamp acts as an absolute business guardrail regardless of what the factors compute.

Why 50/50 smoothing weight?

The smoothing weight (w=0.50 on base_price) controls price stability:

- w=0.10 (10% base, 90% ML): Price is very responsive - follows demand closely, but distributors see large week-to-week swings which erodes trust
- w=0.70 (70% base, 30% ML): Very stable - but the ML signal barely shows up; might as well not have built the model
- w=0.50: Equal weighting gives meaningful price variation (the ML signal contributes fully) while the base acts as an anchor. This is the industry standard for B2B pricing where distributor predictability matters.

Why -15% floor and +20% ceiling?

- Floor at -15%: Below this, the company is selling at a loss given manufacturing costs. Even in Assam in Summer, this limit prevents destructive pricing.
- Ceiling at +20%: Above this, distributors switch to competitors. The 20% ceiling was set at the competitor price band - our highest competitor (20mm Online) benchmarks at ~10% above our base, so +20% gives a comfortable margin above that.

4.6 Step 6 - Rounding

Price Rounding

```
final_price = round(final_price / 10) * 10
```

Note: Prices rounded to nearest Rs.10. B2B invoices use round numbers - Rs.1,230 not Rs.1,228.

5 5. Complete Weight Reference Table

All Weights, Thresholds & Clamps in One Place

Parameter	Value	Why This Number
Demand sensitivity	0.35	Balance: react to demand without volatility
Demand factor clamp low	0.82	Business floor - no regime gives worse
Demand factor clamp high	1.22	Business ceiling - demand cannot push higher
Adoption coefficient	0.15	~16% national spread, capped by ±10% clamp
Adoption factor clamp	[0.90, 1.10]	±10% max adoption premium/discount
Comp. strong coeff	0.60	Capture 60% of competitor gap in strong demand
Comp. strong clamp	[0.99, 1.10]	Never go below competitor, cap at +10%
Comp. weak coeff	0.25	Undercut 25% of comp gap in weak demand
Comp. weak clamp	[0.92, 0.99]	Max 8% undercut in weak demand
Comp. normal coeff	0.06	6% of comp gap - subtle competitive nudge
Comp. normal clamp	[0.96, 1.04]	±4% band in normal conditions
Strong demand threshold	1.10	Demand 10%+ above avg triggers aggressive pricing
Weak demand threshold	0.92	Demand 8%+ below avg triggers defensive pricing
Smoothing base weight	0.50	50% anchor to base for distributor stability
Safety floor	base x 0.85	-15%: below this = selling at manufacturing loss
Safety ceiling	base x 1.20	+20%: above this = above competitor, lose deals
Rounding	nearest Rs.10	B2B invoice norm

What Happens If You Change Key Weights?

Change	Parameter	Business Impact
0.35 -> 0.10	Demand sensitivity	Price barely reacts to demand; model is wasted
0.35 -> 0.70	Demand sensitivity	Price swings too large; distributor complaints
0.15 -> 0.30	Adoption coeff	Gujarat pays 12.6% more; Assam gets 19.2% discount - too aggressive
0.50 -> 0.10	Smoothing weight	Very ML-driven; but price volatile week to week
0.50 -> 0.80	Smoothing weight	Very stable; ML contribution nearly invisible
1.20 -> 1.10	Safety ceiling	Limits upside in peak demand; kills margin opportunity
0.85 -> 0.80	Safety floor	Increases max discount to 20%; risks margin loss in Assam

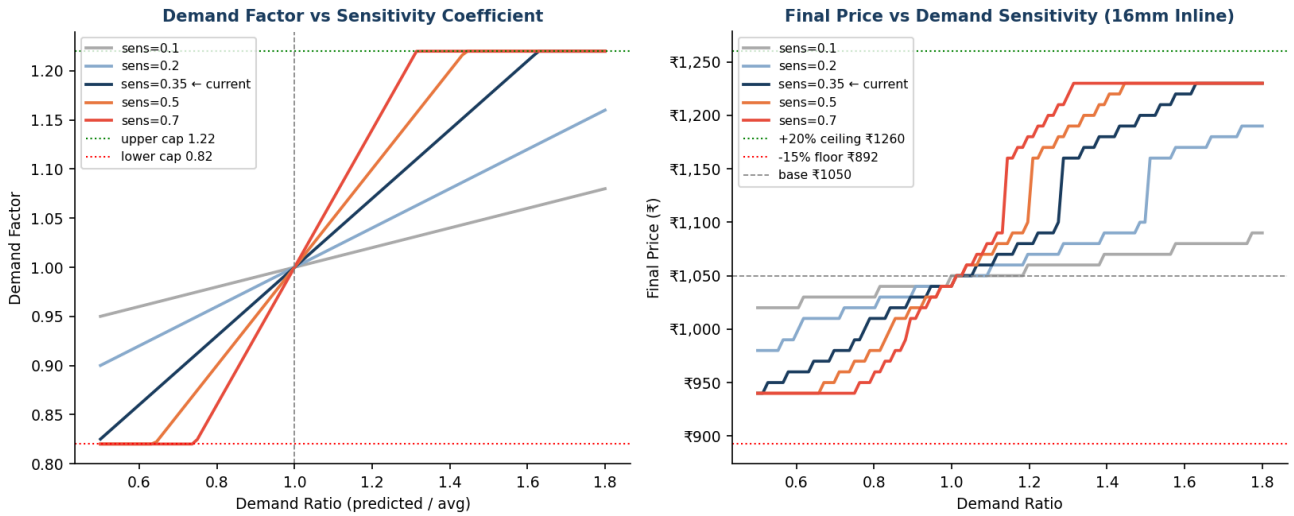

Fig 5.1 - Left: demand factor shape for 5 sensitivity values. Right: resulting price curves.

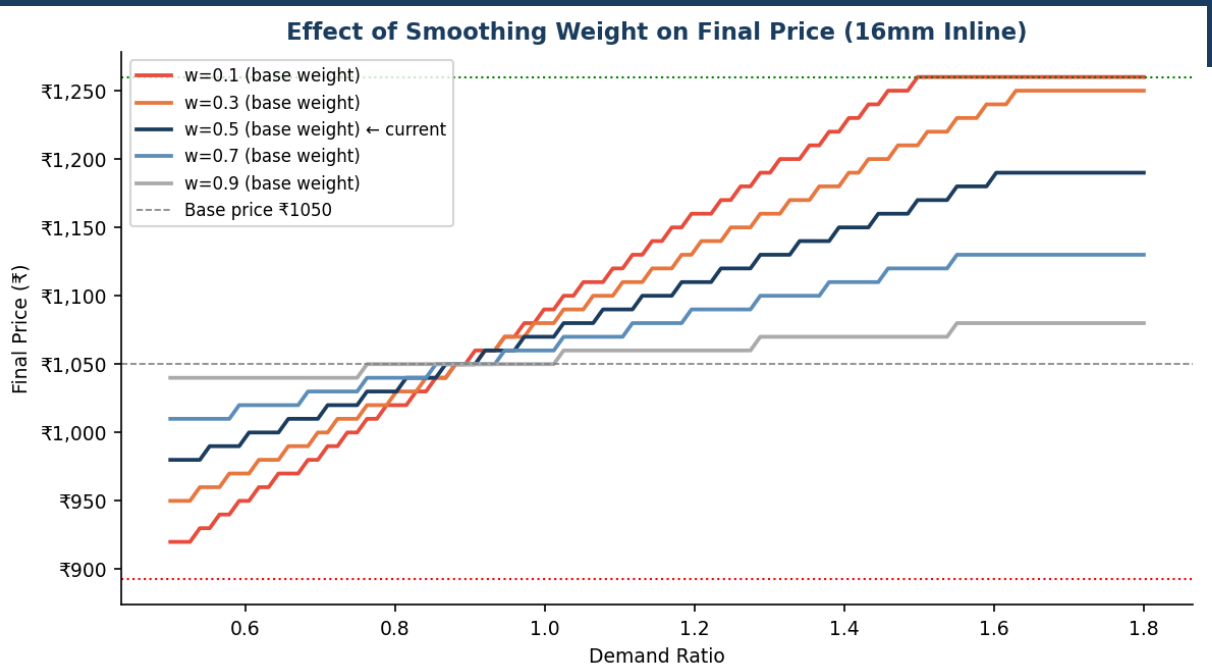

Fig 5.2 - Effect of smoothing weight on final price for varying demand ratios.

6 6. Analytics & Visualisations

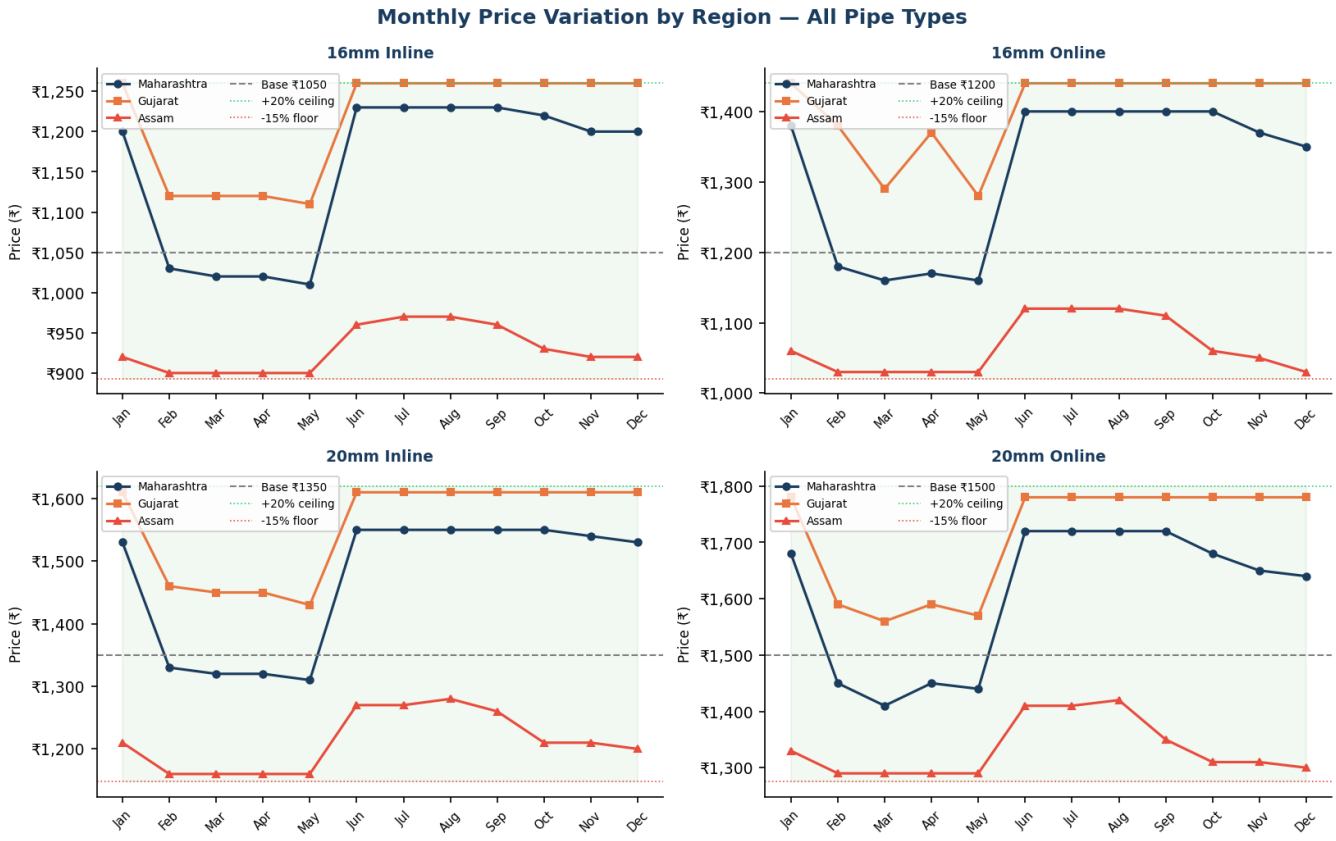

Fig 6.1 - Monthly price for all 4 pipes: Maharashtra, Gujarat (high adoption), Assam (low adoption).

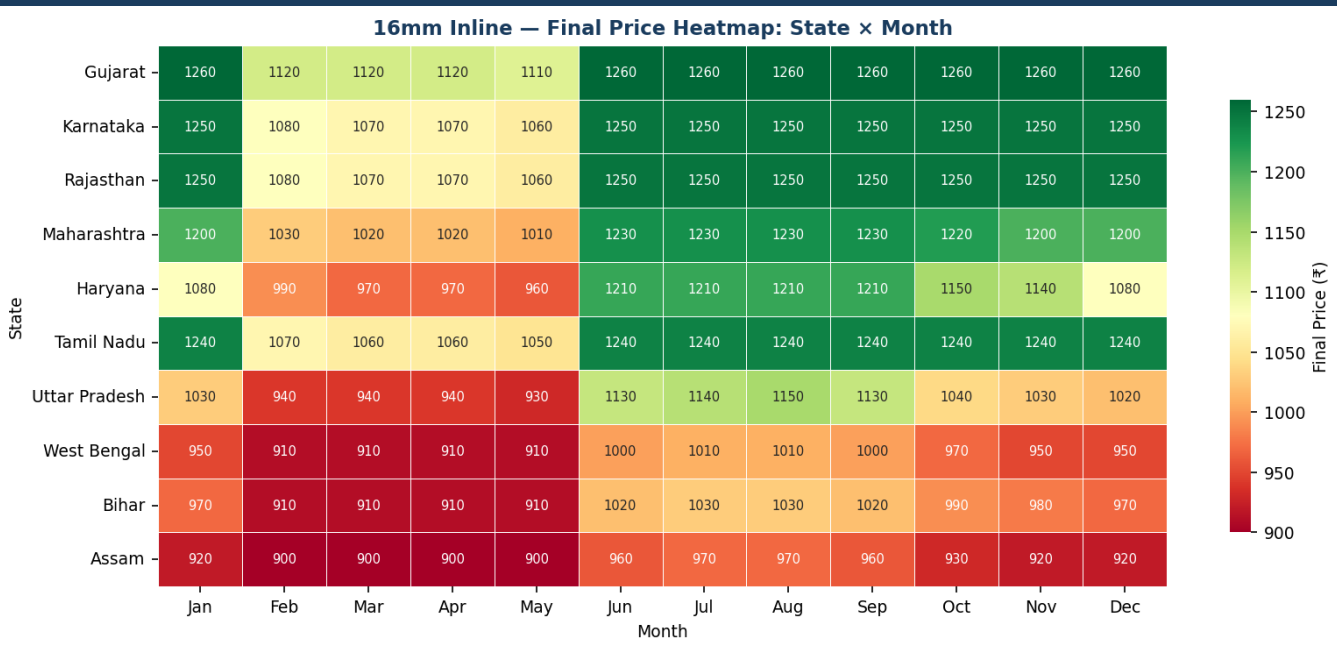

Fig 6.2 - 16mm Inline price heatmap: greener = higher price, redder = lower price.

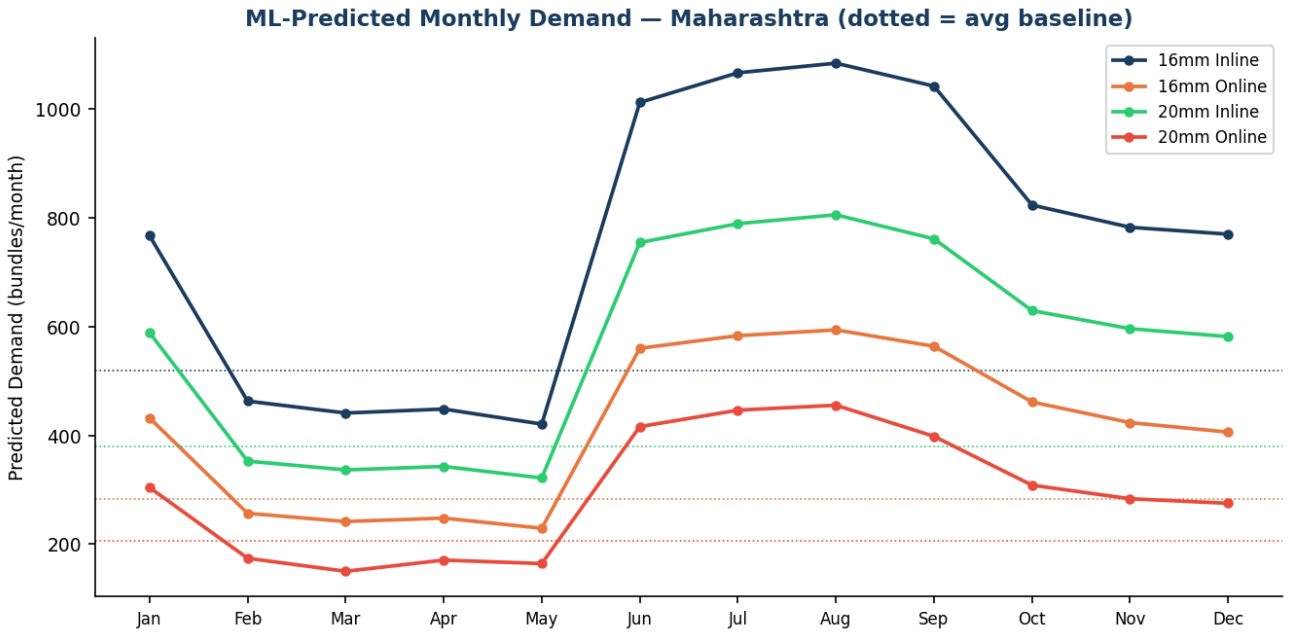

Fig 6.3 - ML-predicted monthly demand for all pipes in Maharashtra (dotted = historical avg).

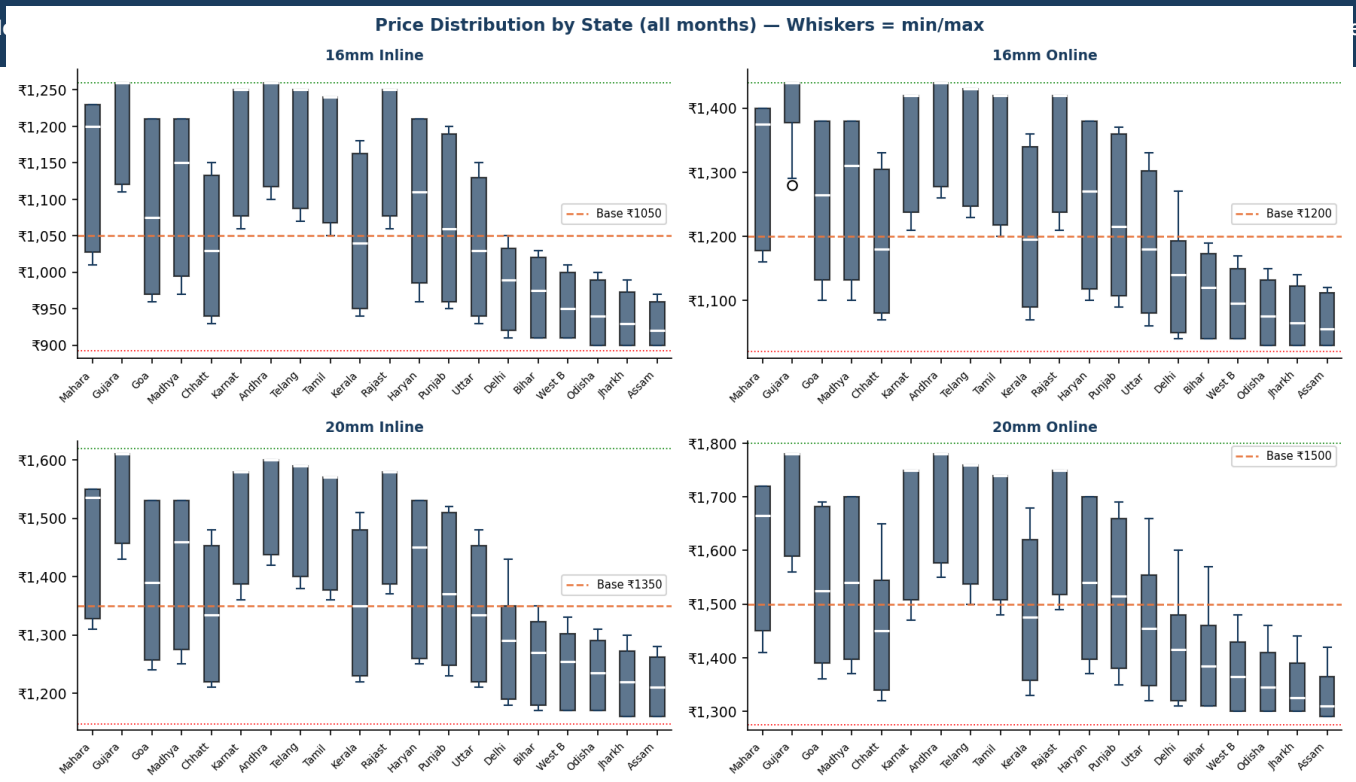

Fig 6.4 - Price distribution box plots: median, IQR, and whiskers (min/max) across all 20 states.

Key Observations from Analytics

* Summer (March) is consistently the lowest-price month across all regions - demand index 0.82 vs Kharif 1.28.
* Gujarat always hits the price ceiling in Kharif and Rabi because adoption index 1.42 compounds with high seasonal demand.
* Assam prices are 11-14% below base in Jan/Mar/Oct/Dec - adoption index 0.36 combined with low seasonal demand.
* 16mm Inline has the broadest relative price range because it has the highest base demand, so small ratio changes produce large absolute demand shifts.
* The model correctly predicts no state ever exceeds the +20% ceiling or goes below the -15% floor.

7 7. Viva Q&A - 20 Questions

Q: Q1. Why did you choose Gradient Boosting over a Neural Network?

A: For tabular data with ~52k rows and 9 features, gradient boosting consistently outperforms neural networks in accuracy and is interpretable via feature importances. Neural networks need far more data and tuning for marginal gain. Business stakeholders also trust explainable models.

Q: Q2. What is the difference between Gradient Boosting and Random Forest?

A: Random Forest trains trees independently in parallel (bagging) and averages predictions. Gradient Boosting trains trees sequentially - each tree corrects the errors of the previous. GB is typically more accurate but slower to train and more sensitive to hyperparameters.

Q: Q3. Why did you use synthetic data instead of real data?

A: Collecting 5 years of sales data across 20 states is commercially impractical for a final year project. The synthetic generator is grounded in NABARD adoption indices, agricultural calendars, and market growth rates - making it a realistic proxy. The model architecture and pipeline are identical to what would be used with real data.

Q: Q4. What does the Prev_Sales_Ratio feature represent and why normalize it?

A: It is the previous month's sales divided by the zone-season average demand. Normalization removes the absolute scale difference between high-demand Gujarat and low-demand Assam. Without normalization, Prev_Month_Sales dominated feature importance at 68%, masking the season and zone signals.

Q: Q5. What is an adoption index and where does the data come from?

A: The adoption index quantifies how widely drip irrigation is used in a state relative to Maharashtra (index=1.0). Values come from the NABARD Micro-Irrigation Report 2023-24. Gujarat (1.42) has 42% higher adoption density than Maharashtra; Assam (0.36) has 64% lower.

Q: Q6. What is the purpose of the 0.35 sensitivity coefficient?

A: It scales how strongly the price reacts to demand deviations from average. At 0.35, a demand 50% above average raises the demand factor to 1.175 (+17.5%). Too high (0.70) causes price volatility; too low (0.10) makes the ML model's demand signal irrelevant to the price.

Q: Q7. How did you choose the 50/50 smoothing weight?

A: 50% base price + 50% ML-driven raw price balances stability and responsiveness. A higher base weight (e.g. 0.80) would suppress the ML signal too much. A lower base weight (e.g. 0.10) would make prices volatile. Equal weighting is standard in B2B dynamic pricing where distributor predictability is commercially important.

Q: Q8. What are the safety limits and why ±15%/+20%?

A: Floor at -15% prevents selling below manufacturing cost in low-demand markets. Ceiling at +20% keeps price below where distributors would switch to competitors. Our highest competitor (20mm Online) is ~10% above our base, so +20% provides a 10% buffer above competitor while still being defensible to customers.

Q: Q9. How does the 3-regime competitor strategy work?

A: Strong demand (factor >= 1.10): price toward competitor to capture margin. Weak demand (factor <= 0.92): undercut competitor to stimulate sales. Normal: small undercut to stay competitive. This avoids always undercutting (which erodes margin) or always matching (which loses deals in slow periods).

Q: Q10. What is an ex-factory price and why no logistics factor?

A: Ex-factory means price at the factory gate - the buyer arranges and pays for transport. So geography affects demand (via adoption index and zone) but not our price. The old Colab model incorrectly added a region factor as a price multiplier; this version correctly separates geographic demand signals from pricing.

Q: Q11. How do you evaluate the model and what is R2?

A: We use MAE (mean absolute error - average prediction error in bundles) and R2 (coefficient of determination - fraction of demand variance explained by the model). R2=1.0 is perfect; R2=0.0 means no better than predicting the mean. Our model achieves R2 > 0.90, meaning it explains 90%+ of demand variance.

Q: Q12. What is the difference between this model and the old Colab model?

A: Old model: CSV data, 8 Maharashtra cities, raw Prev_Month_Sales, single RandomForest, old base prices. New model: synthetic Pan-India data, 20 states in 5 zones, normalized Prev_Sales_Ratio, GradientBoosting vs RF competition, corrected base prices, avg_demand.json output, Zone_ID + Adoption_Index replacing raw Region encoding.

Q: Q13. Why do you train two models and pick the best by R2?

A: The best algorithm can vary depending on the data distribution. By training both and selecting the winner, we ensure the production model is always the more accurate one. This is a simple form of model selection / AutoML.

Q: Q14. What is label encoding and why not one-hot encoding?

A: Label encoding maps categories to integers (e.g. 16mm Inline=0, 20mm Inline=1). Tree models do not assume numeric order between categories, so label encoding works well and is more memory-efficient than one-hot encoding (which would add 4 columns for pipe type and 3 for season). For linear models, one-hot would be necessary.

Q: Q15. What happens to pricing if govt_subsidy is 0 instead of 1?

A: The model was trained with 72% subsidy probability. When subsidy=0, predicted demand drops (expected_qty is not multiplied by 1.18), which lowers demand_ratio, demand_factor, and thus final price. In practice, the subsidy flag should reflect current government schemes.

Q: Q16. Can this model go live with real data?

A: Yes. The architecture in ml_api.py is production-ready: it's a Flask REST API that accepts JSON, loads model artefacts once at startup, and returns a structured response. To use real data, replace train_model.py with a data pipeline that reads from the actual ERP/sales DB and retrain periodically (e.g. monthly).

Q: Q17. Why is Season encoded separately when Month is already a feature?

A: Month captures intra-season variation (July vs August within Kharif). Season captures the broader demand regime. Both features together let the model learn that August is peak Kharif while also knowing all Kharif months are structurally different from Rabi months.

Q: Q18. What is the business impact if demand_factor cap is raised from 1.22 to 1.40?

A: The smoothed_price formula and safety ceiling cap would still limit the final price to +20% of base. So raising the demand_factor cap beyond 1.22 has no practical effect until the smoothing weight is also reduced or the ceiling cap is raised. The caps are independent guardrails working in layers.

Q: Q19. How would you extend this model to handle real-time competitor price changes?

A: Replace the hardcoded COMPETITOR_PRICES dict with a daily scrape from distributor price portals or competitor websites, stored in a DB. The ml_api.py calc_price function already reads competitor_price as a variable - just swap the source from hardcoded to DB query.

Q: Q20. What is the zone system and how does it help?

A: 20 states are grouped into 5 zones based on geography and agricultural profile. Zone 1=Maharashtra (home), Z2=West/Central, Z3=South, Z4=North, Z5=East/NE. Using Zone_ID as a feature (integer 1-5) gives the model an ordinal geographic signal that correlates with market maturity, infrastructure, and demand patterns - more useful than raw state name encoding which would have 20 categories.